AWS Database Blog

# Migrate Oracle's XMLDOM package functions to Amazon Aurora PostgreSQL using JSON format

by Jitendra Kumar and Sai Parthasaradhi | on 05 OCT 2022 | in Advanced (300), Amazon Aurora, PostgreSQL compatible, RDS for PostgreSQL, Technical How-to | Permalink | Comments | Share

Migrating from Oracle to Amazon Aurora PostgreSQL-Compatible Edition or Amazon Relational Database Service (Amazon RDS) for PostgreSQL may be challenging if the application is using the DBMS_XMLDOM package to access XML type objects. This is because the DBMS_XMLDOM package isn't supported in Aurora PostgreSQL or Amazon RDS for PostgreSQL. Moreover, converting procedures to Aurora PostgreSQL or Amazon RDS for PostgreSQL could be a cumbersome and time-consuming process depending on the amount of stored procedures dependent on the DBMS_XMLDOM package.

In this post, we discuss a solution that uses the JSON data type instead of XML to migrate stored procedures from Oracle to PostgreSQL and reduce migration efforts. We chose JSON because JSON is the native format for data in JavaScript, and PostgreSQL has extensive support of JSON compared to XML.

## Overview of solution

The DBMS_XMLDOM package in Oracle is used to access XML type objects and implements the Document Object Model (DOM), an application programming interface for HTML and XML documents.

In this solution, we create the DBMS_XMLDOM schema and wrapper package functions equivalent to Oracle in a PostgreSQL database using the PLV8 extension. PLV8 is a trusted JavaScript language extension for PostgreSQL that you can use for stored procedures, triggers, and more.

This solution uses the JSON data type instead of XML to convert Oracle's DBMS_XMLDOM package to PostgreSQL. Therefore, make sure to complete a thorough assessment of the database and its dependent applications before proceeding with this solution and make sure there are no objections or concerns to using the JSON data type.

## Prerequisites

Assuming that you have an Oracle and a PostgreSQL databases up and running, proceed with the following steps.

Before getting started, you have to create several database objects in PostgreSQL:

1. Create the DBMS_XMLDOM schema in PostgreSQL:

```
CREATE SCHEMA dbms_xmldom AUTHORIZATION postgres;
```

2. Install the plv8 extension:

```
CREATE EXTENSION plv8;
```

PLV8 is a shared library that provides a PostgreSQL procedural language powered by the V8 JavaScript engine.

3. Create the following wrapper functions under the DBMS_XMLDOM schema:
   a. Wrapper function `dbms_xmldom.addelement` (this function adds new element into the document):

```
CREATE OR REPLACE FUNCTION dbms_xmldom.addelement(
  tempjson json,
  mykey text,
  myval text)
    RETURNS json
    LANGUAGE 'plv8'
    COST 100
    VOLATILE PARALLEL UNSAFE
AS $BODY$
let myjson = tempjson;
let tj = Object.values(myjson)[0];
let keyName = mykey;
let val = myval;
if(tj.length === 0){
  let tempObj = {};
  tempObj[keyName] = val;
  myjson[Object.keys(myjson)[0]] = tempObj;
}
```

   b. Wrapper function `dbms_xmldom.getattributes` (this function retrieves the attributes of the node):

```
CREATE OR REPLACE FUNCTION dbms_xmldom.getattributes(
    nodelist json,
    b text)
    RETURNS json
    LANGUAGE 'plv8'
    COST 100
    VOLATILE PARALLEL UNSAFE
AS $BODY$
let response = {};

if (nodelist == null) {
    return null
}
else
    {
        let attrJson=Object.values(nodelist)[0];
        Object.entries(attrJson).forEach(([keyname,val])=>{
                                            if(keyname.indexOf(b) !== -1)
                                            { keyname = keyname.substring(1);
```

   c. Wrapper function `dbms_xmldom.getchildrenbytagname` (returns the children of the document):

```
CREATE OR REPLACE FUNCTION dbms_xmldom.getchildrenbytagname(
  nodelist json,
   x text)
    RETURNS json
    LANGUAGE 'plv8'
    COST 100
    VOLATILE PARALLEL UNSAFE
AS $BODY$
let res = {};
  let nodename = x;
  for (const [keyname, val] of Object.entries(nodelist)) {
    if(keyname === nodename){
    res[keyname] = val;
    return res;
    }
  }
$BODY$;
```

   d. Wrapper function `dbms_xmldom.getelementbytagname` (returns the element in the subtree by the tag name):

```
CREATE OR REPLACE FUNCTION dbms_xmldom.getelementbytagname(
    nodelist json,
    b text)
    RETURNS json
    LANGUAGE 'plv8'
    COST 100
    VOLATILE PARALLEL UNSAFE
AS $BODY$
let response = {};

if (nodelist == null) {
    return null
}
else {
    let attrJson=Object.values(nodelist)[0];
    Object.entries(attrJson).forEach(([keyname,val])=>{
                                        if(keyname.indexOf(b) !== -1)
                                        {response[keyname]=val;}
                                    });
```

   e. Wrapper function `dbms_xmldom.getlength` (retrieves the number of items in the map):

```
CREATE OR REPLACE FUNCTION dbms_xmldom.getlength(
    nodelist json)
    RETURNS integer
    LANGUAGE 'plv8'
    COST 100
    VOLATILE PARALLEL UNSAFE
AS $BODY$
if (nodelist == null)
    return 0
else
    return Object.keys(nodelist).length
$BODY$;
```

   f. Wrapper function `dbms_xmldom.getnodename` (retrieves the name of the node):

```
CREATE OR REPLACE FUNCTION dbms_xmldom.getnodename(
    nodelist json)
    RETURNS text
    LANGUAGE 'plv8'
    COST 100
    VOLATILE PARALLEL UNSAFE
AS $BODY$
return Object.keys(nodelist)[0];
$BODY$;
```

   g. Wrapper function `dbms_xmldom.getnodevalue` (retrieves the value of the node):

```
CREATE OR REPLACE FUNCTION dbms_xmldom.getnodevalue(
    nodelist json)
    RETURNS text
    LANGUAGE 'plv8'
    COST 100
    VOLATILE PARALLEL UNSAFE
AS $BODY$
return Object.values(nodelist)[0];
$BODY$;
```

   h. Wrapper function `dbms_xmldom.item` (retrieves the item given the index in the map):

```
CREATE OR REPLACE FUNCTION dbms_xmldom.item(
    nodelist json,
    index integer)
    RETURNS json
    LANGUAGE 'plv8'
    COST 100
    VOLATILE PARALLEL UNSAFE
AS $BODY$
let Obj = {};
let keyName = Object.keys(nodelist)[index-1];
let val =  Object.values(nodelist)[index-1];
Obj[keyName] = val;
return Obj;
$BODY$;
```

   i. Wrapper function `dbms_xmldom.setattribute` (sets the attribute specified by name; the function is overloaded with TEXT, INTEGER, and DOUBLE PRECISION data types):

```
CREATE OR REPLACE FUNCTION dbms_xmldom.setattribute(
  tempjson json,
  mykey text,
  myval text)
    RETURNS json
    LANGUAGE 'plv8'
    COST 100
    VOLATILE PARALLEL UNSAFE
AS $BODY$
let myjson = tempjson;
let tj = Object.values(myjson)[0];
let keyName = mykey;
let val = myval;
if(tj.length === 0){
  let tempObj = {};
  tempObj[`@${keyName}`] = val;
  myjson[Object.keys(myjson)[0]] = tempObj;
}
```

```
CREATE OR REPLACE FUNCTION dbms_xmldom.setattribute(
  tempjson json,
  mykey text,
  myval integer)
    RETURNS json
    LANGUAGE 'plv8'
    COST 100
    VOLATILE PARALLEL UNSAFE
AS $BODY$
let myjson = tempjson;
let tj = Object.values(myjson)[0];
let keyName = mykey;
let val = myval;
if(tj.length === 0){
  let tempObj = {};
  tempObj[`@${keyName}`] = val;
  myjson[Object.keys(myjson)[0]] = tempObj;
}
```

```
CREATE OR REPLACE FUNCTION dbms_xmldom.setattribute(
  tempjson json,
  mykey text,
  myval double precision)
    RETURNS json
    LANGUAGE 'plv8'
    COST 100
    VOLATILE PARALLEL UNSAFE
AS $BODY$
let myjson = tempjson;
let tj = Object.values(myjson)[0];
let keyName = mykey;
let val = myval;
if(tj.length === 0){
  let tempObj = {};
  tempObj[`@${keyName}`] = val;
  myjson[Object.keys(myjson)[0]] = tempObj;
}
```

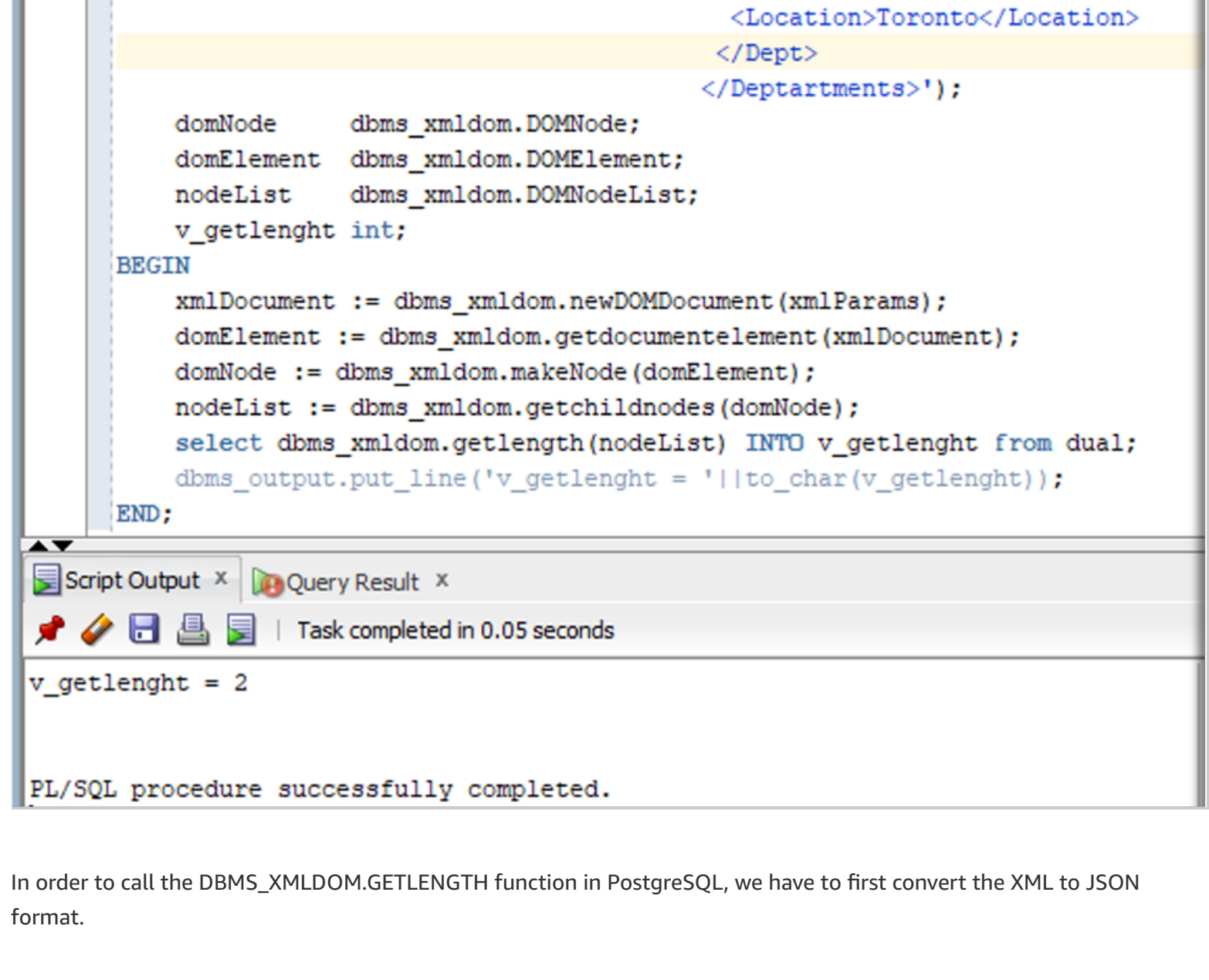
## Example 1: Compare number of nodes

Let's use the DBMS_XMLDOM.GETLENGTH function to return the number of nodes in the list and compare the output between Oracle and PostgreSQL.

Run the following code block in Oracle:

```
DECLARE
    xmlDocument  dbms_xmldom.DOMDocument;
    xmlParams    xmltype := xmltype('<Deptartments>
                                        <Dept DeptID="10">
                                          <Name>Administration</Name>
                                          <Location>Seattle</Location>
                                        </Dept>
                                        <Dept DeptID="20">
                                         <Name>Marketing</Name>
                                         <Location>Toronto</Location>
                                        </Dept>
                                    </Deptartments>');
    domNode      dbms_xmldom.DOMNode;
    domElement   dbms_xmldom.DOMElement;
    nodeList     dbms_xmldom.DOMNodeList;
    v_getlenght int;
BEGIN
    xmlDocument  := dbms_xmldom.newDOMDocument(xmlParams);
    domElement   := dbms_xmldom.getdocumentelement(xmlDocument);
```

You get the following output.



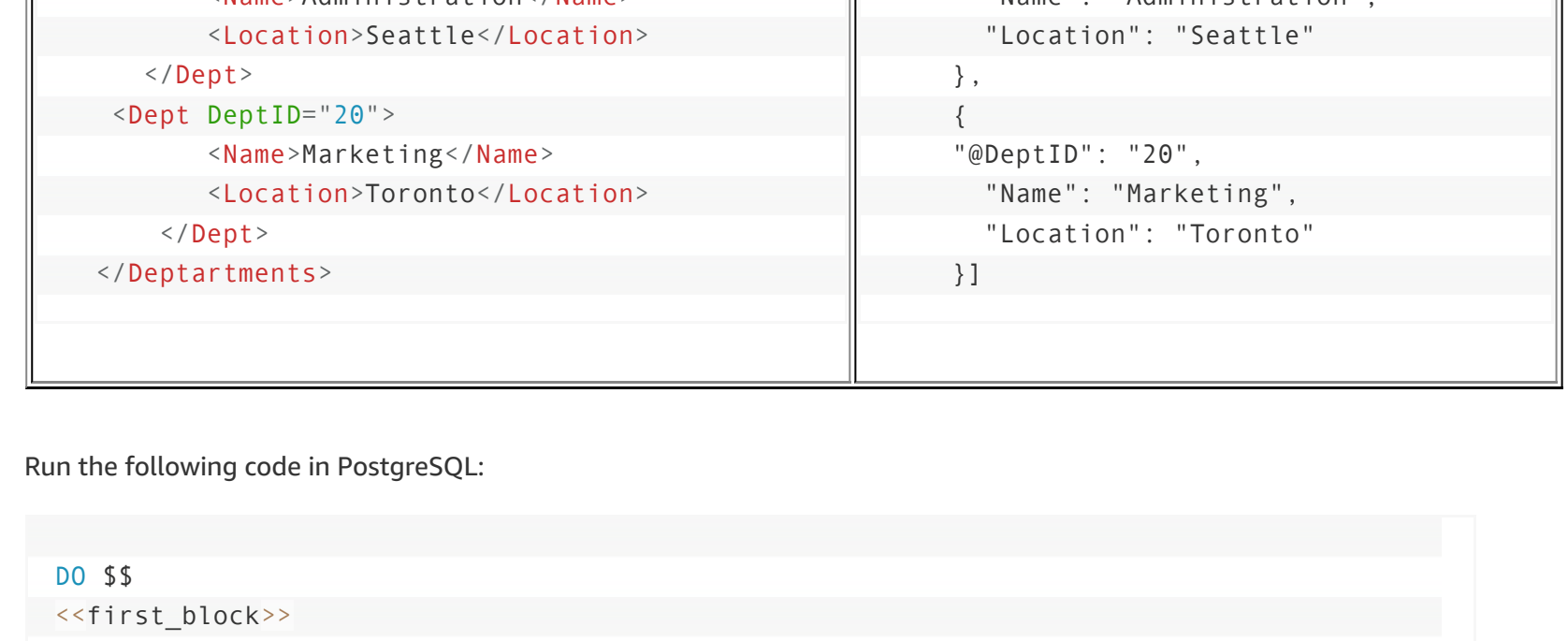
```
DECLARE
    xmlDocument  dbms_xmldom.DOMDocument;
    xmlParams    xmltype := xmltype('<Deptartments>
                                       <Dept DeptID="10">
                                        <Name>Administration</Name>
                                        <Location>Seattle</Location>
                                       </Dept>
                                       <Dept DeptID="20">
                                        <Name>Marketing</Name>
                                        <Location>Toronto</Location>
                                       </Dept>
                                      </Deptartments>');
    domNode     dbms_xmldom.DOMNode;
    domElement  dbms_xmldom.DOMElement;
    nodeList    dbms_xmldom.DOMNodeList;
    v_getlenght int;
BEGIN
    xmlDocument := dbms_xmldom.newDOMDocument(xmlParams);
    domElement := dbms_xmldom.getdocumentelement(xmlDocument);
    domNode  := dbms_xmldom.makeNode(domElement);
    nodeList  := dbms_xmldom.getchildnodes(domNode);
    select dbms_xmldom.getlength(nodeList)  INTO v_getlenght from dual;
    dbms_output.put_line('v_getlenght = '||to_char(v_getlenght));
END;
```

Task completed in 0.05 seconds

```
v_getlenght = 2


PL/SQL procedure successfully completed.
```

In order to call the DBMS_XMLDOM.GETLENGTH function in PostgreSQL, we have to first convert the XML to JSON format.



| XML | JSON |
|---|---|
| `<Deptartments>`<br>`<Dept DeptID="10">`<br>`<Name>Administration</Name>`<br>`<Location>Seattle</Location>`<br>`</Dept>`<br>`<Dept DeptID="20">`<br>`<Name>Marketing</Name>`<br>`<Location>Toronto</Location>`<br>`</Dept>`<br>`</Deptartments>` | `[{`<br>`"@DeptID": "10",`<br>`"Name": "Administration",`<br>`"Location": "Seattle"`<br>`},`<br>`{`<br>`"@DeptID": "20",`<br>`"Name": "Marketing",`<br>`"Location": "Toronto"`<br>`}]` |

Run the following code in PostgreSQL:

```
DO $$
<<first_block>>
DECLARE
    xmlParams  json := '[
                        {
                            "@DeptID": "10",
                            "Name": "Administration",
                            "Location": "Seattle"
                        },
                        {
                            "@DeptID": "20",
                            "Name": "Marketing",
                            "Location": "Toronto"
                        }
                        ]';
    v_getlenght int;
BEGIN
    select dbms_xmldom.getlength(xmlParams) into v_getlenght;
    raise notice 'v_getlenght=%', v_getlenght;
```

You get the following output.

## Example 2: Generate an XML document

Let's create a stored procedure to generate an XML document using the DBMS_XMLDOM package function.

This procedure covers the following DBMS_XMLDOM package functions:

- `dbms_xmldom.getdocumentelement`
- `dbms_xmldom.makeNode`
- `dbms_xmldom.makeNode`
- `dbms_xmldom.createElement`
- `dbms_xmldom.appendChild`
- `dbms_xmldom.setattribute`

The following Oracle procedure code generates the XML document:

```
CREATE OR REPLACE NONEDITIONABLE FUNCTION FN_DateToPlSqlXml (theDate DATE) RETURN VARCHAR2
BEGIN
  RETURN(to_char(theDate, 'yyyy-mm-dd') || 'T' || to_char(theDate, 'hh24:mi:ss'));
END;
```

```
CREATE OR REPLACE PROCEDURE Generate_XML
(
  tranID            NUMBER,
  transDate         DATE,
  transAmt          NUMBER,
  blobText1         VARCHAR2,
  blobText2         VARCHAR2,
  xmlParams         CLOB,
  xmlDocument       IN OUT NOCOPY dbms_xmldom.DOMDocument
) IS
cdataNode         dbms_xmldom.DOMCdataSection;
xmlNode           dbms_xmldom.DOMNode;
nodeList          dbms_xmldom.DOMNodeList;

mydocNode         dbms_xmldom.DOMNode;
mydocElement      dbms_xmldom.DOMElement;

queueNode         dbms_xmldom.DOMNode;
```

The following is the equivalent PostgreSQL procedure code:

```
CREATE OR REPLACE FUNCTION fn_datetoplsqlxml(
    thedate timestamp )
    RETURNS text
    LANGUAGE 'plpgsql'
    COST 100
    VOLATILE PARALLEL UNSAFE
AS $BODY$
BEGIN
    RETURN concat(TO_CHAR(current_date, 'yyyy-mm-dd'),'T', to_char(current_timestamp, 'HH2
END;
$BODY$;
```

```
CREATE OR REPLACE PROCEDURE Generate_XML(
    tranID                  integer,
    transDate            timestamp,
    transAmt              double precision,
    blobText1             varchar(200),
    blobText2             varchar(200),
    xmlparams          text,
    INOUT xmldocument   json)
LANGUAGE 'plpgsql'
AS $BODY$

DECLARE jsndocument JSON;
BEGIN
    IF dbms_xmldom.getlength(xmldocument) > 0 THEN
        jsndocument := xmldocument ;
    ELSEIF xmlParams IS NOT NULL THEN
        jsndocument := json(xmlParams);
    ELSE
```

### Test case 1

In the following Oracle call statement, xmlParams is NULL:

```
set serveroutput on;

declare
    tranID NUMBER           := 11011;
    transDate DATE          := sysdate;
    transAmt NUMBER         := 999.99;
    blobText1 VARCHAR(200)  := 'Credit Card <Refer charges below>';
    blobText2 VARCHAR(200)  := '#SBI Bank Charges @2.25%, Others @3.95%';
    xmlParams CLOB          := NULL;
    xmlDocument dbms_xmldom.DOMDocument;
    vClob        CLOB;
begin
    Generate_XML (tranID,transDate,transAmt,blobText1,blobText2,xmlParams,xmlDocument);

    --print the xml
    dbms_lob.createtemporary(vClob, false);
    dbms_xmldom.writetoclob( doc => xmlDocument, cl => vClob);
    dbms_output.put_line(vClob);
end ;
```

The following screenshot shows our output.

The following is the equivalent PostgreSQL call statement:

```
DO $$
<<first_block>>
    declare tranID integer            := 11011;
    declare transDate timestamp       := current_date;
    declare transAmt double precision  := 999.99;
    declare blobText1 varchar(200)    := 'Credit Card <Refer charges below>';
    declare blobText2 varchar(200)    := '#SBI Bank Charges @2.25%, Others @3.95%';
    declare xmlParams text            := NULL;
    declare xmlDocument json;
    declare jsndocument json;

BEGIN
    call Generate_XML(tranID, transDate, transAmt, blobText1, blobText2, xmlParams, jsndoc
    raise notice '%',jsndocument;

END first_block $$;
```

You get the following output.

### Test case 2

In the following Oracle call statement, xmlParams contains a value:

```
set serveroutput on;

declare
    tranID       NUMBER        := 11022;
    transDate    DATE          := '04-Jul-2022';
    transAmt     NUMBER        := 888.88;
    blobText1    VARCHAR(200)  := 'Credit Card <Refer charges below>';
    blobText2    VARCHAR(200)  := '#SBI Bank Charges @2.25%, Others @3.95%';
    xmlParams    CLOB          := '<ROOT companyID = "9999" >
                                    <PROCESS_QUEUE totalAmt = "1000">
                                    </PROCESS_QUEUE>
                                  </ROOT>';
    xmlDocument dbms_xmldom.DOMDocument;
    vClob       CLOB;
begin
    Generate_XML (tranID,transDate,transAmt,blobText1,blobText2,xmlParams,xmlDocument);

    --print the xml
    dbms_lob.createtemporary(vClob, false);
```

The following screenshot shows our output.

The following code is the equivalent PostgreSQL call statement:

## Resources

- Amazon Aurora
- Amazon DocumentDB
- Amazon DynamoDB
- Amazon ElastiCache
- Amazon Keyspaces (for Apache Cassandra)
- Amazon MemoryDB
- Amazon Neptune
- Amazon Neptune Analytics
- Amazon RDS
- Amazon Timestream
- AWS Database Migration Service

```
DO $$
<<first_block>>
    declare tranID integer              := 11022;
    declare transDate timestamp         := '04-Jul-2022';
    declare transAmt double precision   := 888.88;
    declare blobText1 varchar(200)      := 'Credit Card <Refer charges below>';
    declare blobText2 varchar(200)      := '#SBI Bank Charges @2.25%, Others @3.95%';
    declare xmlParams text              := '{"@companyID": "9999",
                                            "PROCESS_QUEUE": {"@totalAmt": "1000"}
                                           }';

    declare xmlDocument json;
    declare jsndocument json;

BEGIN
    call Generate_XML(tranID, transDate, transAmt, blobText1, blobText2,
        xmlParams, jsndocument);
    raise notice '%',jsndocument;

END first_block $$;
```

We get the following output.

## Example 3: Process the XML document

Let's create another stored procedure to process the XML document. This procedure accepts the XML document created by the `Generate_XML` procedure as the input parameter, reads all the items, and stores them in the `TRANSACTION_DETAILS` table.

This procedure covers the following DBMS_XMLDOM package functions:

- `dbms_xmldom.getattributes`
- `dbms_xmldom.getnodevalue`
- `dbms_xmldom.item`
- `dbms_xmldom.getelementbytagname`
- `dbms_xmldom.getlength`

Let's create the TRANSACTION_DETAILS table in our databases.

**Oracle**

```
CREATE  TABLE TRANSACTION_DETAILS
(
    tranID      NUMBER NOT NULL PRIMARY KEY,
    transDate   DATE NOT NULL,
    transAmt    NUMBER,
    totalAmt    NUMBER,
    blobText1   CHAR,
    blobText2   CHAR,
    modified_time timestamp default sysdate
);
```

**PostgreSQL**

```
CREATE  TABLE TRANSACTION_DETAILS
(
    tranID         INTEGER NOT NULL PRIMARY KEY,
    transDate   DATE NOT NULL,
    transAmt    DOUBLE PRECISION,
    totalAmt    DOUBLE PRECISION,
    blobText1   TEXT,
    blobText2   TEXT,
    modified_time TIMESTAMP default CURRENT_TIMESTAMP
);
```

The following Oracle code creates the `Process_XML` procedure:

```
CREATE OR REPLACE PROCEDURE Process_XML
    (
        xmlDocument     IN dbms_xmldom.DOMDocument,
        errCode         OUT NUMBER,
        errMessage      OUT VARCHAR2
    ) AS

xactdocElement       dbms_xmldom.DOMElement;
xactdocNode          dbms_xmldom.DOMNode;
queueNode            dbms_xmldom.DOMNode;
blob1Node            dbms_xmldom.DOMNode;
blob2Node            dbms_xmldom.DOMNode;

attributeList         dbms_xmldom.DOMNamedNodeMap;
attributeIndex        NUMBER;
attributeNode         dbms_xmldom.DOMNode;

nodeList              dbms_xmldom.DOMNodeList;
```

The following procedure code is the PostgreSQL equivalent:

```
CREATE OR REPLACE PROCEDURE public.process_xml
(
    xmldocument             JSON,
    INOUT err_code          INTEGER,
    INOUT err_message       TEXT
)
LANGUAGE 'plpgsql'
AS $BODY$
DECLARE
    jsndocument             JSON;
    attributelist           JSON;
    attributeNode           JSON;
    nodelist                JSON;

    v_tranID                INTEGER;
    v_transDate             TIMESTAMP;
    v_transAmt              DOUBLE PRECISION;
    v_totalAmt              DOUBLE PRECISION;
    v_blobText1             VARCHAR(200);
```

### Test case 3

In the following Oracle call statement, xmlParams is NULL:

```
set serveroutput on;

declare
    tranID          NUMBER        := 11011;
    transDate       DATE          := sysdate;
    transAmt        NUMBER        := 999.99;
    blobText1       VARCHAR(200)  := 'Credit Card <Refer charges below>';
    blobText2       VARCHAR(200)  := '#SBI Bank Charges @2.25%, Others @3.95%';
    xmlParams       CLOB          := NULL;
    xmlDocument     dbms_xmldom.DOMDocument;
    o_errCode       NUMBER;
    o_errMessage    VARCHAR2(64);
begin
    Generate_XML (tranID,transDate,transAmt,blobText1,blobText2,xmlParams,xmlDocument);
    Process_XML (xmlDocument,o_errCode,o_errMessage);

    DBMS_OUTPUT.PUT_LINE('o_errCode:= ' || o_errCode || ', o_errMessage := ' || o_errMes
```

We get the following output.

The following code is the PostgreSQL equivalent:

```
DO $$
<<first_block>>
    declare tranID integer              := 11011;
    declare transDate timestamp         := current_date;
    declare transAmt double precision   := 999.99;
    declare blobText1 varchar(200)      := 'Credit Card <Refer charges below>';
    declare blobText2 varchar(200)      := '#SBI Bank Charges @2.25%, Others @3.95%';
    declare xmlParams text              := NULL;
    declare xmlDocument json;
    declare jsndocument json;

    declare o_err_code integer;
    declare o_err_message text;

BEGIN
    call Generate_XML(tranID, transDate, transAmt, blobText1, blobText2,
                    xmlParams, jsndocument);
    call Process_XML (jsndocument, o_err_code,o_err_message);
    raise notice 'o_err_code = %, o_errmessage=%',o_err_code,o_err_message ;
```

We get the following output.

### Test case 4

In the following Oracle call statement, xmlParams contains a value:

```
set serveroutput on;

declare
    tranID            NUMBER              := 11022;
    transDate       DATE                    := '04-Jul-2022';
    transAmt        NUMBER              := 888.88;
    blobText1       VARCHAR(200)  := 'Credit Card <Refer charges below>';
    blobText2       VARCHAR(200)  := '#SBI Bank Charges @2.25%, Others @3.95%';
    xmlParams     CLOB                    := '<ROOT companyID = "9999" >
                                                <PROCESS_QUEUE totalAmt
                                                </PROCESS_QUEUE>
                                            </ROOT>';
    xmlDocument     dbms_xmldom.DOMDocument;
    o_errCode             NUMBER;
    o_errMessage    VARCHAR2(64);
begin
    Generate_XML (tranID,transDate,transAmt,blobText1,blobText2,xmlParams,xmlDocument);
    Process_XML (xmlDocument,o_errCode,o_errMessage);
```

The following screenshot shows our output.

To verify the table record in Oracle, run the following query:

```
SELECT * FROM TRANSACTION_DETAILS;
```

You get the following output.

The following call statement is the PostgreSQL equivalent:

```
DO $$
<<first_block>>
    declare tranID integer              := 11022;
    declare transDate timestamp         := '04-Jul-2022';
    declare transAmt double precision   := 888.88;
    declare blobText1 varchar(200)      := 'Credit Card <Refer charges below>';
    declare blobText2 varchar(200)      := '#SBI Bank Charges @2.25%, Others @3.95%';
    declare xmlParams text              := '{"@companyID": "9999",
                                            "PROCESS_QUEUE": {"@totalAmt": "1000"}
                                           }';
    declare jsndocument json;
    declare o_err_code integer;
    declare o_err_message text;

BEGIN
    call Generate_XML(tranID, transDate, transAmt, blobText1, blobText2,
                    xmlParams, jsndocument);
    call Process_XML (jsndocument, o_err_code,o_err_message);
    raise notice 'o_err_code = %, o_errmessage=%',o_err_code,o_err_message ;
```

We get the following output.

Verify the table record in PostgreSQL with the following query:

```
SELECT * FROM TRANSACTION_DETAILS;
```

You get the following output.

## Conclusion

In this post, we showed you a solution to migrate Oracle's DBMS_XMLDOM package functions to Aurora PostgreSQL or Amazon RDS for PostgreSQL using the JSON data type.

If you have any questions or comments, share your thoughts in the comment section.

---

### About the Authors

**Jitendra Kumar** is a Lead Database Migration Consultant with AWS Professional Services. He helps customers migrate and modernize workloads in the AWS Cloud, with a special focus on modern application architectures and development best practices.

**Sai Parthasaradhi** is a Database Migration Specialist with AWS Professional Services. He works closely with customers to help them migrate and modernize their databases on AWS.

Like Share

## Comments

Log in to comment

Learn
What Is AWS?
What Is Cloud Computing?
What Is Agentic AI?
Cloud Computing Concepts Hub
AWS Cloud Security
What's New
Blogs
Press Releases

Resources
Getting Started
Training
AWS Trust Center
AWS Solutions Library
Architecture Center
Product and Technical FAQs
Analyst Reports
AWS Partners

Developers
Builder Center
SDKs & Tools
.NET on AWS
Python on AWS
Java on AWS
PHP on AWS
JavaScript on AWS

Help
Contact Us
File a Support Ticket
AWS re:Post
Knowledge Center
AWS Support Overview
Get Expert Help
AWS Accessibility
Legal

Back to top

Amazon is an Equal Opportunity Employer: Minority / Women / Disability / Veteran / Gender Identity / Sexual Orientation / Age.

Privacy Site terms Cookie Preferences © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.